

Comprehensive Guide to `<string.h>`

Functions in C

This document provides a detailed overview of all standard functions in the C `<string.h>` library, including syntax, explanations, example code, comparisons, and guidance on choosing between similar functions. It is designed for students learning C programming, covering practical use cases, edge cases, and connections to related concepts like pointers, arrays, file handling, and dynamic memory allocation.

Introduction

The `<string.h>` library in C provides functions for manipulating null-terminated strings (arrays of characters ending with `'\0'`). These functions handle tasks like copying, concatenating, comparing, searching, and measuring strings, as well as memory operations for raw byte arrays. This guide covers each function, grouped by category, with examples and comparisons to help you choose the right function for your needs.

String Manipulation Functions

1. `strlen`

- **Syntax:** `size_t strlen(const char *str);`
- **Description:** Returns the length of the null-terminated string `str`, excluding the null terminator (`'\0'`).
- **Return Value:** Number of characters in `str` (type `size_t`, an unsigned integer).
- **Use Case:** Determine the length of a string for iteration, allocation, or validation.
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    const char *str = "Hello";
    size_t len = strlen(str);
    printf("Length of '%s': %zu\n", str, len); // Output: Length of 'Hello': 5
}
```

```

    return 0;
}

```

- **Edge Cases:**

- **Unterminated String:** If `str` lacks `'\0'`, causes undefined behavior (UB).
- **NULL Pointer:** Passing `NULL` causes UB.

- **Connection:** Relates to your arrays and pointers questions, as it counts characters in a `char` array.

2. strcpy

- **Syntax:** `char *strcpy(char *dest, const char *src);`
- **Description:** Copies the null-terminated string `src` (including `'\0'`) to `dest`.
- **Return Value:** Pointer to `dest`.
- **Use Case:** Copy a string to a new buffer.
- **Example:**

```

#include <stdio.h>
#include <string.h>
int main() {
    char dest[20];
    const char *src = "Hello";
    strcpy(dest, src);
    printf("Copied string: %s\n", dest); // Output: Copied string: He
    return 0;
}

```

- **Edge Cases:**

- **Insufficient Space:** If `dest` is too small, causes buffer overflow (UB).
- **NULL Pointers:** Passing `NULL` for `dest` or `src` causes UB.
- **Overlap:** If `dest` and `src` overlap, causes UB.

- **Connection:** Used with dynamic allocation (from your `malloc` / `realloc` questions) to copy strings safely.

3. strncpy

- **Syntax:** `char *strncpy(char *dest, const char *src, size_t n);`
- **Description:** Copies up to `n` characters from `src` to `dest`. If `src` is shorter than `n`, pads `dest` with `'\0'`. If `src` is longer, no `'\0'` is added to `dest`.
- **Return Value:** Pointer to `dest`.

- **Use Case:** Safer copying with a size limit to prevent buffer overflow.
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char dest[10];
    const char *src = "HelloWorld";
    strncpy(dest, src, 5);
    dest[5] = '\\0'; // Manually add null terminator
    printf("Copied string: %s\\n", dest); // Output: Copied string: He
    return 0;
}
```

- **Edge Cases:**
 - **No Null Terminator:** If `src` has `n` or more characters, `dest` may not be null-terminated.
 - **NULL Pointers:** Passing `NULL` causes UB.
 - **Overlap:** If `dest` and `src` overlap, causes UB.
- **Comparison with `strcpy`:**
 - **Choose `strncpy`:** When you need to limit the number of copied characters to avoid overflow.
 - **Choose `strcpy`:** When you're sure `dest` is large enough and want simpler code.
 - **Why?:** `strncpy` is safer but requires manual null termination if `src` is long; `strcpy` is faster but risky.
- **Connection:** Safer for file input (from your file handling question) to prevent overflows.

4. `strcat`

- **Syntax:** `char *strcat(char *dest, const char *src);`
- **Description:** Appends `src` (including '\\0') to the end of `dest` , overwriting `dest` 's original '\\0' .
- **Return Value:** Pointer to `dest` .
- **Use Case:** Concatenate two strings.
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char dest[20] = "Hello";
    const char *src = ", World!";
    strcat(dest, src);
}
```

```

    printf("Concatenated: %s\n", dest); // Output: Concatenated: Hell
    return 0;
}

```

- **Edge Cases:**
 - **Insufficient Space:** If `dest` can't hold both strings, causes buffer overflow.
 - **NULL Pointers:** Passing `NULL` causes UB.
 - **Overlap:** If `dest` and `src` overlap, causes UB.
- **Connection:** Useful for building strings from file data (from your file handling question).

5. `strncat`

- **Syntax:** `char *strncat(char *dest, const char *src, size_t n);`
- **Description:** Appends up to `n` characters from `src` to `dest`, adding `'\0'` at the end.
- **Return Value:** Pointer to `dest`.
- **Use Case:** Safer concatenation with a size limit.
- **Example:**

```

#include <stdio.h>
#include <string.h>
int main() {
    char dest[20] = "Hello";
    const char *src = ", World!";
    strncat(dest, src, 7); // Copies ", World"
    printf("Concatenated: %s\n", dest); // Output: Concatenated: Hell
    return 0;
}

```

- **Edge Cases:**
 - **Insufficient Space:** Must ensure `dest` has room for `n` characters plus `'\0'`.
 - **NULL Pointers:** Passing `NULL` causes UB.
 - **Overlap:** If `dest` and `src` overlap, causes UB.
- **Comparison with `strcat`:**
 - **Choose `strncat`:** When you need to limit appended characters for safety.
 - **Choose `strcat`:** When `dest` is guaranteed to have enough space and simplicity is preferred.
 - **Why?:** `strncat` prevents overflow but requires size calculation; `strcat` is faster but risky.
- **Connection:** Safer for dynamic string building (from your dynamic allocation question).

6. strcmp

- **Syntax:** `int strcmp(const char *str1, const char *str2);`
- **Description:** Compares two null-terminated strings lexicographically (based on ASCII values).
- **Return Value:**
 - `< 0`: `str1` is less than `str2` (e.g., "apple" < "banana").
 - `0`: `str1` equals `str2`.
 - `> 0`: `str1` is greater than `str2`.
- **Use Case:** Sort strings or check equality.
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    const char *str1 = "apple";
    const char *str2 = "banana";
    int result = strcmp(str1, str2);
    printf("Comparison: %d\n", result); // Output: Comparison: -1 (apple < banana)
    return 0;
}
```

- **Edge Cases:**
 - **NULL Pointers:** Passing `NULL` causes UB.
 - **Unterminated Strings:** Causes UB.
- **Connection:** Used in sorting strings (from your selection sort and callbacks questions).

7. strncmp

- **Syntax:** `int strncmp(const char *str1, const char *str2, size_t n);`
- **Description:** Compares up to `n` characters of `str1` and `str2` or until a `'\0'` is reached.
- **Return Value:** Same as `strcmp`.
- **Use Case:** Compare string prefixes or limit comparison length.
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    const char *str1 = "apple";
    const char *str2 = "apricot";
    int result = strncmp(str1, str2, 3);
    printf("Comparison: %d\n", result); // Output: Comparison: 0 (apple & apricot start with 'app')
}
```

```

    return 0;
}

```

- **Edge Cases:**
 - **NULL Pointers:** Passing `NULL` causes UB.
 - **Unterminated Strings:** Safe up to `n` characters, but UB if no `'\0'` within `n`.
- **Comparison with `strcmp`:**
 - **Choose `strncmp`:** For prefix comparison or when strings may be long/unterminated.
 - **Choose `strcmp`:** For full string comparison when null termination is guaranteed.
 - **Why?:** `strncmp` is safer and more flexible; `strcmp` is simpler and faster for full strings.
- **Connection:** Useful in parsing file data (from your file handling question).

8. `strchr`

- **Syntax:** `char *strchr(const char *str, int c);`
- **Description:** Finds the first occurrence of character `c` (cast to `char`) in `str`.
- **Return Value:** Pointer to the found character or `NULL` if not found.
- **Use Case:** Search for a delimiter or specific character.
- **Example:**

```

#include <stdio.h>
#include <string.h>
int main() {
    const char *str = "Hello, World!";
    char *pos = strchr(str, ',');
    if (pos) {
        printf("Found at: %s\n", pos); // Output: Found at: , World!
    }
    return 0;
}

```

- **Edge Cases:**
 - **NULL Pointer:** Passing `NULL` for `str` causes UB.
 - **Character Not Found:** Returns `NULL`.
- **Connection:** Useful for parsing strings from files (from your file handling question).

9. `strrchr`

- **Syntax:** `char *strrchr(const char *str, int c);`
- **Description:** Finds the last occurrence of character `c` in `str`.

- **Return Value:** Pointer to the last occurrence or `NULL` if not found.
- **Use Case:** Find the last delimiter (e.g., file extension).
- **Example:**

```
#include <stdio.h>
#include <string.h>

int main() {
    const char *str = "file.txt.txt";
    char *pos = strrchr(str, '.');
    if (pos) {
        printf("Last extension: %s\n", pos); // Output: Last extensio
    }
    return 0;
}
```

- **Edge Cases:** Same as `strchr`.
- **Comparison with `strchr`:**
 - **Choose `strrchr`:** When you need the last occurrence (e.g., file extensions).
 - **Choose `strchr`:** For the first occurrence (e.g., first delimiter).
 - **Why?:** `strrchr` searches backward; `strchr` is forward, affecting which match you get.
- **Connection:** Relates to string manipulation (from your strings question).

10. `strstr`

- **Syntax:** `char *strstr(const char *haystack, const char *needle);`
- **Description:** Finds the first occurrence of substring `needle` in `haystack`.
- **Return Value:** Pointer to the start of the substring or `NULL` if not found.
- **Use Case:** Search for a pattern in a string.
- **Example:**

```
#include <stdio.h>
#include <string.h>

int main() {
    const char *haystack = "Hello, World!";
    const char *needle = "World";
    char *pos = strstr(haystack, needle);
    if (pos) {
        printf("Found: %s\n", pos); // Output: Found: World!
    }
    return 0;
}
```

- **Edge Cases:**
 - **NULL Pointers:** Passing `NULL` causes UB.
 - **Empty Needle:** Returns `haystack` (implementation-defined).
- **Connection:** Useful for text processing in files (from your file handling question).

11. `strtok`

- **Syntax:** `char *strtok(char *str, const char *delim);`
- **Description:** Tokenizes `str` by splitting it into substrings separated by characters in `delim`. Maintains internal state for subsequent calls.
- **Return Value:** Pointer to the next token or `NULL` if no more tokens.
- **Use Case:** Parse delimited strings (e.g., CSV).
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "apple,banana,cherry";
    const char *delim = ",";
    char *token = strtok(str, delim);
    while (token) {
        printf("Token: %s\n", token);
        token = strtok(NULL, delim);
    }
    return 0;
}
```

- **Output:**

```
Token: apple
Token: banana
Token: cherry
```

- **Edge Cases:**
 - **NULL Pointer:** Passing `NULL` for `delim` causes UB; `str` can be `NULL` on subsequent calls.
 - **Thread Safety:** Not thread-safe due to internal state.
 - **Modifies String:** Replaces delimiters with `'\0'`.
- **Connection:** Useful for parsing file input (from your file handling question).

12. `strtok_r` (POSIX, not standard C)

- **Syntax:** `char *strtok_r(char *str, const char *delim, char **saveptr);`
- **Description:** Reentrant version of `strtok`, using `saveptr` to store state explicitly.
- **Return Value:** Same as `strtok`.
- **Use Case:** Tokenize strings in multithreaded or nested parsing scenarios.
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "apple,banana,cherry";
    const char *delim = ",";
    char *saveptr;
    char *token = strtok_r(str, delim, &saveptr);
    while (token) {
        printf("Token: %s\n", token);
        token = strtok_r(NULL, delim, &saveptr);
    }
    return 0;
}
```

- **Edge Cases:** Similar to `strtok`, but safer for threading.
- **Comparison with `strtok`:**
 - **Choose `strtok_r`:** For thread-safe or nested tokenization.
 - **Choose `strtok`:** For single-threaded, simpler code.
 - **Why?:** `strtok_r` is reentrant; `strtok` is not thread-safe.
- **Note:** Not in ISO C, but common in POSIX systems.

Memory Manipulation Functions

13. `memcpy`

- **Syntax:** `void *memcpy(void *dest, const void *src, size_t n);`
- **Description:** Copies `n` bytes from `src` to `dest`. Does not handle null terminators.
- **Return Value:** Pointer to `dest`.
- **Use Case:** Copy raw memory blocks (e.g., structures, arrays).
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
```

```

char src[] = "Hello";
char dest[10];
memcpy(dest, src, 6); // Copy 5 chars + '\0'
printf("Copied: %s\n", dest); // Output: Copied: Hello
return 0;
}

```

- **Edge Cases:**
 - **Overlap:** Causes UB if `dest` and `src` overlap.
 - **NULL Pointers:** Passing `NULL` causes UB.
- **Connection:** Useful for copying dynamic arrays (from your dynamic allocation question).

14. memmove

- **Syntax:** `void *memmove(void *dest, const void *src, size_t n);`
- **Description:** Copies `n` bytes from `src` to `dest`, handling overlapping regions safely.
- **Return Value:** Pointer to `dest`.
- **Use Case:** Copy memory when `dest` and `src` may overlap.
- **Example:**

```

#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "Hello, World!";
    memmove(str + 7, str, 6); // Overlap: copy "Hello" to "World"
    printf("Moved: %s\n", str); // Output: Moved: Hello, Hello!
    return 0;
}

```

- **Edge Cases:**
 - **NULL Pointers:** Passing `NULL` causes UB.
- **Comparison with `memcpy`:**
 - **Choose `memmove`:** When `dest` and `src` may overlap.
 - **Choose `memcpy`:** For non-overlapping regions, as it's faster.
 - **Why?:** `memmove` handles overlap safely but is slower; `memcpy` assumes no overlap for efficiency.
- **Connection:** Safer for array manipulation (from your arrays question).

15. memcmp

- **Syntax:** `int memcmp(const void *s1, const void *s2, size_t n);`

- **Description:** Compares the first `n` bytes of `s1` and `s2`.
- **Return Value:**
 - `< 0` : `s1` is less than `s2`.
 - `0` : Equal.
 - `> 0` : `s1` is greater than `s2`.
- **Use Case:** Compare raw memory blocks (e.g., structures).
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[] = "apple";
    char s2[] = "apricot";
    int result = memcmp(s1, s2, 3);
    printf("Comparison: %d\n", result); // Output: Comparison: 0
    return 0;
}
```

- **Edge Cases:**
 - **NULL Pointers:** Passing `NULL` causes UB.
- **Comparison with `strcmp` / `strncmp` :**
 - **Choose `memcmp` :** For non-string memory or exact byte comparison.
 - **Choose `strcmp` / `strncmp` :** For null-terminated strings.
 - **Why?:** `memcmp` compares raw bytes; `strcmp` / `strncmp` stop at `'\0'`.

16. `memset`

- **Syntax:** `void *memset(void *s, int c, size_t n);`
- **Description:** Sets the first `n` bytes of `s` to the value `c` (cast to `unsigned char`).
- **Return Value:** Pointer to `s`.
- **Use Case:** Initialize memory (e.g., set array to zero).
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[10];
    memset(str, 'A', 5);
    str[5] = '\0';
    printf("Set: %s\n", str); // Output: Set: AAAAA
```

```

    return 0;
}

```

- **Edge Cases:**
 - **NULL Pointer:** Passing `NULL` causes UB.
- **Connection:** Useful for initializing dynamic arrays (from your dynamic allocation question).

17. `memchr`

- **Syntax:** `void *memchr(const void *s, int c, size_t n);`
- **Description:** Finds the first occurrence of `c` (cast to `unsigned char`) in the first `n` bytes of `s`.
- **Return Value:** Pointer to the found byte or `NULL` if not found.
- **Use Case:** Search for a byte in memory.
- **Example:**

```

#include <stdio.h>
#include <string.h>
int main() {
    const char *str = "Hello";
    void *pos = memchr(str, 'l', 5);
    if (pos) {
        printf("Found at: %s\n", (char *)pos); // Output: Found at: l
    }
    return 0;
}

```

- **Edge Cases:**
 - **NULL Pointer:** Passing `NULL` causes UB.
- **Comparison with `strchr`:**
 - **Choose `memchr`:** For non-string memory or limited search length.
 - **Choose `strchr`:** For null-terminated strings.
 - **Why?:** `memchr` searches `n` bytes; `strchr` stops at `'\0'`.

Miscellaneous Functions

18. `strerror`

- **Syntax:** `char *strerror(int errnum);`
- **Description:** Returns a string describing the error code `errnum` (e.g., from `errno`).

- **Return Value:** Pointer to a static string (do not modify).
- **Use Case:** Print error messages for system calls.
- **Example:**

```
#include <stdio.h>
#include <string.h>
#include <errno.h>

int main() {
    errno = EACCES;
    printf("Error: %s\n", strerror(errno)); // Output: Error: Permission denied
    return 0;
}
```

- **Edge Cases:**
 - **Invalid `errno`:** Returns "Unknown error" or similar (implementation-defined).
 - **Thread Safety:** May not be thread-safe due to static storage.
- **Connection:** Useful for file handling errors (from your file handling question).

19. `strspn`

- **Syntax:** `size_t strspn(const char *str, const char *accept);`
- **Description:** Returns the length of the initial segment of `str` consisting only of characters in `accept`.
- **Return Value:** Number of matching characters.
- **Use Case:** Parse strings with valid characters.
- **Example:**

```
#include <stdio.h>
#include <string.h>

int main() {
    const char *str = "123abc";
    const char *accept = "0123456789";
    size_t len = strspn(str, accept);
    printf("Digit prefix length: %zu\n", len); // Output: Digit prefix length: 3
    return 0;
}
```

- **Edge Cases:**
 - **NULL Pointers:** Passing `NULL` causes UB.
- **Connection:** Useful for input validation in files (from your file handling question).

20. `strcspn`

- **Syntax:** `size_t strcspn(const char *str, const char *reject);`
- **Description:** Returns the length of the initial segment of `str` containing no characters from `reject`.
- **Return Value:** Number of non-matching characters.
- **Use Case:** Find position of a delimiter.
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    const char *str = "Hello,World";
    const char *reject = ",";
    size_t len = strcspn(str, reject);
    printf("Before delimiter: %zu\n", len); // Output: Before delimit
    return 0;
}
```

- **Edge Cases:** Same as `strspn`.
- **Comparison with `strspn`:**
 - **Choose `strspn`:** To find characters in a set.
 - **Choose `strcspn`:** To find characters not in a set.
 - **Why?:** `strspn` includes; `strcspn` excludes.
- **Connection:** Complements `strtok` for parsing (from your file handling question).

21. `strpbrk`

- **Syntax:** `char *strpbrk(const char *str, const char *accept);`
- **Description:** Finds the first character in `str` that matches any character in `accept`.
- **Return Value:** Pointer to the found character or `NULL` if none found.
- **Use Case:** Locate any delimiter.
- **Example:**

```
#include <stdio.h>
#include <string.h>
int main() {
    const char *str = "Hello,World";
    const char *accept = ",;!";
    char *pos = strpbrk(str, accept);
    if (pos) {
```

```

        printf("Found delimiter: %c\n", *pos); // Output: Found delir
    }
    return 0;
}

```

- **Edge Cases:** Same as `strchr`.
- **Comparison with `strchr`:**
 - Choose `strpbrk`: To search for any of multiple characters.
 - Choose `strchr`: For a single character.
 - **Why?:** `strpbrk` is more flexible; `strchr` is simpler for one character.

Comparisons and Choosing Functions

- **Copying (`strcpy` vs. `strncpy` vs. `memcpy` vs. `memmove`):**
 - Use `strcpy` for simple string copying with known sizes.
 - Use `strncpy` for size-limited copying, but ensure null termination.
 - Use `memcpy` for non-string memory or known sizes without overlap.
 - Use `memmove` for overlapping regions.
- **Concatenation (`strcat` vs. `strncat`):**
 - Use `strcat` for simple concatenation with enough space.
 - Use `strncat` for safety with size limits.
- **Comparison (`strcmp` vs. `strncmp` vs. `memcmp`):**
 - Use `strcmp` for full string comparison.
 - Use `strncmp` for prefix or limited comparison.
 - Use `memcmp` for raw memory comparison.
- **Searching (`strchr` vs. `strrchr` vs. `strstr` vs. `memchr` vs. `strpbrk`):**
 - Use `strchr` for first character match.
 - Use `strrchr` for last character match.
 - Use `strstr` for substring search.
 - Use `memchr` for byte search in memory.
 - Use `strpbrk` for any of multiple characters.
- **Tokenization (`strtok` vs. `strtok_r`):**
 - Use `strtok` for single-threaded parsing.
 - Use `strtok_r` for thread-safe or nested parsing.
- **Length and Span (`strlen` vs. `strspn` vs. `strcspn`):**
 - Use `strlen` for string length.
 - Use `strspn` for prefix of accepted characters.
 - Use `strcspn` for prefix excluding rejected characters.

Practical Example: Combining `<string.h>` Functions

This example reads strings from a file, sorts them case-insensitively, and writes tokens to another file, using multiple `<string.h>` functions.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define INITIAL_SIZE 10
#define GROW_FACTOR 2
#define LOG(fmt, ...) fprintf(stderr, "[%s:%d] " fmt "\n", __FILE__, __LINE__, \
    __VA_ARGS__)

int compare_strings(const void *a, const void *b) {
    const char *str1 = *(const char **)a;
    const char *str2 = *(const char **)b;
    char c1, c2;
    do {
        c1 = tolower(*str1++);
        c2 = tolower(*str2++);
    } while (c1 == c2 && c1 != '\0');
    return c1 - c2; // Custom strcmp
}

char **read_strings(FILE *in, int *n) {
    char **arr = (char **)malloc(INITIAL_SIZE * sizeof(char *));
    int capacity = INITIAL_SIZE;
    *n = 0;
    char buffer[100];
    while (fscanf(in, "%99s", buffer) == 1) {
        if (*n == capacity) {
            capacity *= GROW_FACTOR;
            char **temp = (char **)realloc(arr, capacity * sizeof(char *));
            if (!temp) {
                LOG("Reallocation failed");
                for (int i = 0; i < *n; i++) free(arr[i]);
                free(arr);
                return NULL;
            }
            arr = temp;
        }
        arr[*n] = (char *)malloc(strlen(buffer) + 1);
```



```

        strcpy(arr[*n], buffer);
        (*n)++;
    }
    return arr;
}

int main() {
    FILE *in = fopen("input.txt", "r");
    if (!in) {
        LOG("Cannot open input.txt");
        return 1;
    }

    int n;
    char **arr = read_strings(in, &n);
    fclose(in);
    if (!arr) return 1;

    qsort(arr, n, sizeof(char *), compare_strings);

    FILE *out = fopen("output.txt", "w");
    if (!out) {
        LOG("Cannot open output.txt");
        for (int i = 0; i < n; i++) free(arr[i]);
        free(arr);
        return 1;
    }

    for (int i = 0; i < n; i++) {
        char *token = strtok(arr[i], ",;");
        while (token) {
            fprintf(out, "%s\n", token);
            token = strtok(NULL, ",;");
        }
    }

    for (int i = 0; i < n; i++) free(arr[i]);
    free(arr);
    fclose(out);
    printf("Processed strings on June 16, 2025, 11:24 PM IST\n");
    return 0;
}

```

- **Input File (input.txt):** Banana,apple Cherry;date Fig

- **Output File (output.txt):**

```
apple
Banana
Cherry
date
Fig
```

- **Functions Used:** `strlen`, `strcpy`, `strtok`, `qsort` with custom `strcmp`, `malloc`, `realloc`, `free`.

Edge Cases and Pitfalls

- **Null Pointers:** Most functions cause UB with `NULL` arguments.
- **Unterminated Strings:** Cause UB in string functions.
- **Buffer Overflows:** Avoid with `strncpy`, `strncat`, or size checks.
- **Overlap:** Use `memmove` instead of `memcpy` for overlapping regions.
- **Thread Safety:** `strtok`, `strerror` are not thread-safe; use `strtok_r` or alternatives.
- **Memory Leaks:** Free dynamically allocated strings (from your dynamic allocation question).

Exam Questions

Beginner

1. What is the difference between `strcpy` and `strncpy`?

- **Answer:** `strcpy` copies a full string; `strncpy` copies up to `n` characters, may not null-terminate.

2. Write code to find the last '.' in a filename.

```
char *pos = strrchr("file.txt", '.'); // Returns ".txt"
```

Intermediate

3. Write a program to tokenize a CSV line using `strtok`.

```
char line[] = "apple,banana,cherry";
char *token = strtok(line, ",");
while (token) {
    printf("%s\n", token);
}
```

```
token = strtok(NULL, ",");  
}
```

4. Compare `memcpy` and `memmove`.

- **Answer:** `memcpy` is faster but fails with overlap; `memmove` handles overlap safely.

Advanced

5. Write a program to sort strings from a file using `qsort` and a custom comparator.

- See practical example above.

6. Explain the risks of `strtok` and how `strtok_r` mitigates them.

- **Answer:** `strtok` is not thread-safe due to static state; `strtok_r` uses explicit state for thread safety.

Tips for Use

- **Safety:** Prefer `strncpy`, `strncat`, `strncmp` for bounded operations.
- **Performance:** Use `strcpy`, `strcat`, `strcmp` for known-safe inputs.
- **Memory:** Use `memcpy` / `memmove` for non-strings, `memset` for initialization.
- **Parsing:** Combine `strtok`, `strchr`, `strstr` for file processing.
- **Sorting:** Use `qsort` with custom `strcmp` for flexibility.